

Careerbytecode internship

Week 2 assignment

1. Explain the problems developers faced before Docker and how containerization solved them.

- Environment inconsistency - "Works on my machine" - dev, QA, and prod environments differed, so apps broke on deploy
- Dependency conflicts - two apps needing library/runtime versions couldn't safely share a server
- Heavy, slow VMs, VMs were the only isolation option - heavy (full guest OS), slow to boot, expensive to run at scale
- Painful deployments - Deployments were manual, script-heavy, and hard to roll back
- Poor server utilization - Because apps couldn't safely share machines, servers often ran at 10 - 15% capacity

How containerization solved this:

- Image = app + dependencies + config, built once from a dockerfile. The same image runs identically on a laptop, CI, and production. Environment drift disappears.
- Process-level isolation - using linux kernel features (namespaces, cgroups) instead of a guest OS. containers start in milliseconds and use MBs, not GBs - so you can pack many isolated apps on one host
- Immutable, versioned artifacts - Deploy = pull image + run. Rollback = run the previous tag. No configuration drift on servers.
- Portability - Any host with a container runtime can run the image - cloud, on-prem, any distro
- Foundation for orchestration - standardized units made kubernetes possible : scheduling, scaling and self-healing operate on containers, not hand-configured servers.

2. Differentiate between a Virtual Machine and a Container in terms of architecture, boot time, resource usage, isolation, and portability.

Aspect	Virtual machine	Container
Architecture	Runs on a hypervisor, each VM has a full guest OS + kernel on top of virtualized hardware	Runs on a container engine, shares the host OS kernel, packages only app + dependencies
Boot time	Minutes (boots an entire OS)	Milliseconds to seconds (just starts a process)
Resource usage	Heavy - GBs of disk/RAM per VM due to guest OS overhead	Lightweight - MBs, no OS overhead, higher density per host
Isolation	Strong - hardware-level isolation, separate kernels, better for untrusted workloads	Weaker - process-level isolation via namespaces/cgroups, shared kernel is a larger attack surface
Portability	Limited , larger images tied to	High - small standarized images (OCI)

	hypervisor formats	run on any host with a container runtime
--	--------------------	--

3. Explain the complete Docker image lifecycle — from writing a Dockerfile to running a container.

- Write the Dockerfile
- Build the image - `docker build -t appname:version`
- Tag and push - `docker tag appname:version` , `docker push user/appname:version`
- Pull on target machine - `docker pull user/appname:version`
- Run the container - `docker run -d -p 8080:80 user/appname:version`
- Manage the container lifecycle - `docker ps`, `docker logs`, `docker exec -it container-name shell` , etc...

4. Write the docker commands to perform the following

- Build an image from a Dockerfile with the tag `myapp:v1`

```
varun@DESKTOP-3JFO0FS: /r × + v
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/
RUN sed -i 's/listen 80;/listen 8080;/' /etc/nginx/conf.d/default.conf
EXPOSE 8080
CMD ["nginx", "-g", "daemon off;"]
~
~
```

```
varun@DESKTOP-3JFO0FS:/mnt/c/Users/admin/week-2-assignment/docker-files$ sudo vi Dockerfile
varun@DESKTOP-3JFO0FS:/mnt/c/Users/admin/week-2-assignment/docker-files$ docker build -t my-app:v1 .
[+] Building 16.7s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 214B
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 139B
=> [1/3] FROM docker.io/library/nginx:alpine@sha256:4a73073bd557c65b759505da037898b61f1be6cbcc3c2c3a
=> => resolve docker.io/library/nginx:alpine@sha256:4a73073bd557c65b759505da037898b61f1be6cbcc3c2c3a
=> => sha256:46519e7231d2eb5604df229beb44d59719a489eaa7aca52982535a010b07a9ed 20.31MB / 20.31MB
=> => sha256:d0008c891db48b5f526d914bce9e8d889fe1a9d1f08291ae03fe97f871726f38 1.21kB / 1.21kB
=> => sha256:390dc935348d8070e695fbaae2a4bb114fb9e69c59f628e7576036ee9d5244c9 1.40kB / 1.40kB
=> => sha256:46f977ee452f4399c208714afa034868d6056864f8a0cf3c643ab143dd802c80 404B / 404B
=> => sha256:62bec68d7c31c4c8a19d812d84da5f7748e54690c037979945b6c5b6c924b142 957B / 957B
=> => sha256:1223f016b4e4a2c21f7c49d4837fbfd47a9da6436b511690ca1e582fc2810d59 627B / 627B
=> => sha256:3cd534fe98c64d68a1f4f1c83abb8d5cba7ecfd7be88e592389929d12e6253da 1.89MB / 1.89MB
=> => sha256:55afalecc21d2bb5e5045f32dafee56272ffdd89860bac26f6c32123439af26a4 3.85MB / 3.85MB
=> => extracting sha256:55afalecc21d2bb5e5045f32dafee56272ffdd89860bac26f6c32123439af26a4
=> => extracting sha256:3cd534fe98c64d68a1f4f1c83abb8d5cba7ecfd7be88e592389929d12e6253da
=> => extracting sha256:1223f016b4e4a2c21f7c49d4837fbfd47a9da6436b511690ca1e582fc2810d59
=> => extracting sha256:62bec68d7c31c4c8a19d812d84da5f7748e54690c037979945b6c5b6c924b142
=> => extracting sha256:46f977ee452f4399c208714afa034868d6056864f8a0cf3c643ab143dd802c80
=> => extracting sha256:d0008c891db48b5f526d914bce9e8d889fe1a9d1f08291ae03fe97f871726f38
=> => extracting sha256:390dc935348d8070e695fbaae2a4bb114fb9e69c59f628e7576036ee9d5244c9
=> => extracting sha256:46519e7231d2eb5604df229beb44d59719a489eaa7aca52982535a010b07a9ed
=> [2/3] COPY index.html /usr/share/nginx/html/
=> [3/3] RUN sed -i 's/listen 80;/listen 8080;/' /etc/nginx/conf.d/default.conf
=> => exporting image
=> => exporting layers
=> => exporting manifest sha256:cbc35aa115ae66261f1b9b893473f5fc109f100433d484bace92f281e99447e2
=> => exporting config sha256:39640a20f0a323cc4b3b9d5175f117243938c402353039c63bcd4dc326a3e665
=> => exporting attestation manifest sha256:d4c52dff5738edeb0e9176d630fb400632934a464c83faa0f8eda5c69
=> => exporting manifest list sha256:323fb0ff43c1788c6f6cab0c9dbad06877ccf3c5b9182fc9d00913b5e1be47436
=> => naming to docker.io/library/my-app:v1
=> => unpacking to docker.io/library/my-app:v1
```

b. List all the images

```
varun@DESKTOP-3JFO0FS:/mnt/c/Users/admin/week-2-assignment/docker-files$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
gcr.io/k8s-minikube/kicbase:v0.0.50	b97074569ae9	1.94GB	545MB	
gcr.io/k8s-minikube/kicbase@sha256:eb4fec0e8ad70adf8e6436f195cc429825ff8b5f95afcd5d8d9deb576f3e93	eb4fec0e8ad	1.94GB	545MB	
hello-world:latest	c3be1cc1aa5	25.9kB	9.49kB	
my-app:v1	323fb0ff43c1	92.7MB	26.1MB	

c. Run the container in detached mode on port 8080

```
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/docker-files$ docker run -d -p 8080:8080 --name myapplication my-app:v1
9a221cbb3885a6dbe7113c3765a44bde27d8a8783193f911c68528e99637356a
```

d. View container logs

```
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/docker-files$ docker logs myapplication
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/07/21 04:35:13 [notice] 1#1: using the "epoll" event method
2026/07/21 04:35:13 [notice] 1#1: nginx/1.31.3
2026/07/21 04:35:13 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
2026/07/21 04:35:13 [notice] 1#1: OS: Linux 6.18.33.2-microsoft-standard-WSL2
2026/07/21 04:35:13 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:1048576
2026/07/21 04:35:13 [notice] 1#1: start worker processes
2026/07/21 04:35:13 [notice] 1#1: start worker process 30
2026/07/21 04:35:13 [notice] 1#1: start worker process 31
2026/07/21 04:35:13 [notice] 1#1: start worker process 32
2026/07/21 04:35:13 [notice] 1#1: start worker process 33
2026/07/21 04:35:13 [notice] 1#1: start worker process 34
2026/07/21 04:35:13 [notice] 1#1: start worker process 35
2026/07/21 04:35:13 [notice] 1#1: start worker process 36
2026/07/21 04:35:13 [notice] 1#1: start worker process 37
```

e. Remove the image

```
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/docker-files$ docker rmi my-app:v1
Error response from daemon: conflict: unable to delete my-app:v1 (must be forced) - container 9a221cbb3885 is using its referenced image 323fb0ff43c1
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/docker-files$ docker stop myapplication
myapplication
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/docker-files$ docker rm myapplication
myapplication
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/docker-files$ docker rmi my-app:v1
Untagged: my-app:v1
Deleted: sha256:323fb0ff43c1788c6f6cab0c6bad06877ccf3c5b9182fc9d00913b5e1be47436
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/docker-files$
```

5. Explain the difference between the CMD and ENTRYPOINT instructions in a Dockerfile with examples.

DESCRIPTION	CMD	ENTRYPOINT
Purpose	Default command/args	Fixed executable
docker run img args	Replaces CMD	Appends arg to ENTRYPOINT
Override the instruction itself	Just pass new command	Requires - -entrypoint flag
Example	In Dockerfile <pre>FROM ubuntu CMD ["echo", "Hello world"]</pre> While executing the Dockerfile <pre>docker run sample-image docker run sample-image ls -l</pre> → CMD is replaced entirely → runs ls -l	In Dockerfile <pre>FROM ubuntu ENTRYPOINT ["echo"]</pre> While executing the Dockerfile <pre>docker run sample-image docker run sample-image hello devops</pre> → “hello devops” argos are appended to echo

6. **Write a multi-instruction Dockerfile for a web application and explain each line. Include FROM, WORKDIR, COPY, RUN, EXPOSE, and CMD**

```
varun@DESKTOP-3JFOOFS: /r  ×  +  v
# Base image - start from official Node.js on Alpine Linux (small size)
FROM node:20-alpine

# Set working directory inside the container - all following commands run here
WORKDIR /app

# Copy dependency files first (better layer caching)
COPY package*.json ./

# Install dependencies - runs at BUILD time, creates a layer
RUN npm install --production

# Copy the rest of the application code
COPY . .

# Document which port the app listens on (doesn't publish it - that's -p at run time)
EXPOSE 3000

# Default command - runs at CONTAINER START time (only one CMD per Dockerfile)
CMD ["node", "server.js"]
```

7. **What is docker networking?**

Docker networking is how containers communicate with each other, with the host and with the external world. Docker creates virtual networks and assigns containers to them, the network driver determines the behaviour.

a. Bridge (default)

- i. Containers live in a private network inside your machine, each with its own IP.
- ii. “ docker run -d nginx #joins the default bridge network, docker network create mynet #custom bridge network, it’s better, docker run -d -network=mynet -name web nginx “

b. Host

- i. Container uses your machine’s network directly, no separation. Whatever port the app uses inside is the port on your machine. Fast , but no privacy and ports can clash.
- ii. docker run -d -network=host nginx

c. None

- i. No network at all. The container is completely cut off.
- ii. docker run -d -network=none nginx

d. Overlay

- i. One network stretched across multiple machines. A container on server A talks to container on server B as if they’re on the same machine. Used in cluster (swarm/kubernetes)
- ii. docker network create -d overlay mynet #requires swarm mode

8. What is Docker Compose? Write a simple `docker-compose.yml` that runs two services (for example, a web app and a database).

Docker compose is a tool for defining and running multi-container applications with a single YAML file. Instead of running multiple long docker run commands with networks, volumes, and env vars, you can declare everything in `docker-compose.yml` and start it all with one command

Docker compose up

Docker-compose.yml file (webapp + database)

```
version: "3.8"

services:
  web:
    build: .                # build from Dockerfile in current dir
    ports:
      - "8080:3000"        # host:container
    environment:
      - DB_HOST=db          # service name works as hostname
      - DB_USER=root
      - DB_PASSWORD=root
    depends_on:
      - db                  # start db first

  db:
    image: postgres:16-alpine # use prebuilt image
    environment:
      - POSTGRES_USER=root
      - POSTGRES_PASSWORD=root
      - POSTGRES_DB=root
    volumes:
      - db_data:/var/lib/postgresql/data # persist data

volumes:
  db_data:                  # named volume survives container removal
  ~
  ~
  ~
  ~
```

```
varun@DESKTOP-3JFOOFS:/mnt/c/Users/admin/week-2-assignment/docker-files$ docker compose up
WARN[0000] /mnt/c/Users/admin/week-2-assignment/docker-files/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it
to avoid potential confusion
[+] up 18/1s
✓Image postgres:16-alpine Pulled                                41.7s
[+] Building 2.8s (10/10) FINISHED
=> [internal] load local bake definitions                        0.0s
=> => reading from stdin 5488                                     0.0s
=> [internal] load build definition from Dockerfile              0.1s
=> => transferring dockerfile: 214B                                0.0s
=> [internal] load metadata for docker.io/library/nginx:alpine  1.8s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                       0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 31B                                       0.0s
=> [1/3] FROM docker.io/library/nginx:alpine@sha256:4a73873bd557c65b759505da037898b61f1be6cbcc3c2c3aeac22d2a470c1752  0.1s
=> => resolve docker.io/library/nginx:alpine@sha256:4a73873bd557c65b759505da037898b61f1be6cbcc3c2c3aeac22d2a470c1752  0.1s
=> => CACHED [2/3] COPY index.html /usr/share/nginx/html/         0.0s
=> CACHED [3/3] RUN sed -i 's/listen 80;/listen 8080;/' /etc/nginx/conf.d/default.conf  0.0s
=> => exporting to image                                           0.3s
=> => exporting layers                                             0.0s
=> => exporting manifest sha256:01cb63695f60477e128ad12ced6799dc4f60f755bf245238d4b1b05187a163f2  0.0s
=> => exporting config sha256:0ff5995e64969b7539419642ee669a3a3a1f92903626a575513b7262239b4d39  0.0s
=> => exporting attestation manifest sha256:0f801a15d8921dd1d5f22da3a7432ad3b2316d26fb1e80fd380c25688fb68e4b  0.1s
=> => exporting manifest list sha256:6e69aa01c57a5822e6733a20877058aa3f23117d4858011217f099d489151b7e  0.0s
=> => naming to docker.io/library/docker-files-web:latest        0.0s
[+] up 18/1s
✓Image postgres:16-alpine Pulled                                41.7s
✓Image docker-files-web Built                                    3.0s
✓Network docker-files_default Created                           0.1s
✓Volume docker-files_db_data Created                            0.0s
✓Container docker-files-db-1 Created                             0.3s
✓Container docker-files-web-1 Created                             0.1s
Attaching to db-1, web-1
db-1 | The files belonging to this database system will be owned by user "postgres".
db-1 | This user must also own the server process.
db-1 |
db-1 | The database cluster will be initialized with locale "en_US.utf8".
db-1 | The default database encoding has accordingly been set to "UTF8".
```

9. What is a Docker Registry? Explain the steps to push an image to Docker Hub.

A docker registry is a storage and distribution system for docker images - like github but for images instead of code. You can push images to it and pull them from any machine.

Types:

- 1.dockerhub → the default public registry (hub.docker.com)
- 2.cloud registries → AWS ECR, AZURE ACR, etc..
- 3.private/self-hosted → docker's own registry image

Steps to push an image to docker hub:

- a. docker login # create dockerhub account, then login from CLI, enter username, passwd
- b. docker build -t myapp:version . # build your own image
- c. docker tag myapp:version username/appname:version # tag it with dockerhub username
- d. docker push username/appname:version #push to dockerhub registry
- e. docker pull username/appname:version #verify- pull from dockerhub to any machine

10. Research any three Docker best practices for writing efficient and secure images.

- a. Use minimal base images → big images will have more packages, more security holes and slow downloads, so use minimal base images like alpine versions
- b. Use multi-stage builds → build the app in one stage, copy only the final output to clean stage. Compilers and dev tools stay out of the final image - smaller and safer.
- c. Don't run as root → by default containers run as root , dangerous if hacked. Create a normal user, RUN adduser -S appuser, USER appuser, add the lines in dockerfile for adding user to container

11. Explain the difference between the Control Plane and the Worker Node in Kubernetes, and list the components that run on each.

Control plane	Worker node
The control plane makes decisions about the cluster like scheduling, desired state, responding to events	Worker nodes actually run the workloads of your pods, containers
Components of control plane are kube-apiserver, etcd, kube-scheduler, kube-controller-manager, cloud-controller-manager	Components of worker nodes are kubelet, kube-proxy, container runtime

12. What is the role of etcd in a Kubernetes cluster? Why is it important?

Etcd is the cluster's single source of truth , a consistent, distributed key-value store that holds all cluster state, every object you create like deployments, pods, services, configmaps, secrets, etc.. when you run kubectl get, the API server is ultimately reading from etcd and using Raft consensus for consistency and fault tolerance.

Why it's important:

- a. It is the cluster state
- b. Consistency guarantees
- c. Watch mechanism
- d. Availability math

13. Write a Deployment YAML manifest that runs 3 replicas of an **nginx** container, and write the command to apply it.

```
varun@DESKTOP-3JFOOFS: /r  X + v
apiVersion: apps/v1
kind: Deployment
metadata:
  name: three-nginx
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
        ports:
        - containerPort: 80
~
~

varun@DESKTOP-3JFOOFS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ kubectl apply -f three-nginx.yaml
deployment.apps/three-nginx created
varun@DESKTOP-3JFOOFS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ kubectl get pods
NAME                                READY   STATUS             RESTARTS   AGE
three-nginx-fd956d49d-c7tmq         0/1     ContainerCreating   0           10s
three-nginx-fd956d49d-ngp96         0/1     ContainerCreating   0           10s
three-nginx-fd956d49d-qxr766       0/1     ContainerCreating   0           10s
varun@DESKTOP-3JFOOFS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ |
```

14. Explain how a Service enables communication with Pods. Write a NodePort Service manifest for a Deployment.

The problem services solve: pods are ephemeral. When a deployment recreates a pod (crash, rollout, scale event), the new pod gets a new IP. you can't hardcode pod IPs anywhere , they change constantly.

A service gives you a stable virtual IP (clusterIP) and DNS name that stays fixed while the pods behind it come and go.

How it works:

- Label selector → the service doesn't track pods by IP or name. It selects them by labels (app:name). Any pod matching the selector becomes a backend automatically.
- Endpoints → kubernetes continuously maintains an endpoints object listing the IPs of all ready pods matching the selector. Pod dies → removed from the list. New pod passes its readiness probe → added.
- Kube-proxy → on every node, kube-proxy programs iptables/IPVS rules so that traffic to the service's clusterIP get DNAT'd to one of the backend pod IPs. this gives you load balancing across replicas for free
- DNS → CoreDNS gives the service a name, other pods just call the name and never think about the IPs.

```
varun@DESKTOP-3JFOOFS: /r  ×  +  ▾
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30080
    protocol: TCP

varun@DESKTOP-3JFOOFS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ sudo vi three-nginx.yaml
varun@DESKTOP-3JFOOFS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ sudo vi service.yaml
varun@DESKTOP-3JFOOFS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ kubectl apply -f service.yaml
service/nginx-service created
varun@DESKTOP-3JFOOFS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes           ClusterIP   10.96.0.1     <none>         443/TCP          4h14m
nginx-service        NodePort    10.100.183.17 <none>         80:30080/TCP     13s
```

15. What is an Ingress in Kubernetes? Explain how it is different from a NodePort or LoadBalancer Service.

An ingress is an http/https routing layer that sits in front of your service. Instead of exposing each service separately, you expose one entry point and define the rules for which traffic goes where - based on hostname or URL path.

One IP , one set of rules, many services behind it.

How it differs from nodeport and loadbalancer

Description	NodePort	LoadBalancer	Ingress
Layer	L4(TCP/UDP)	L4(TCP/UDP)	L7(HTTP/HTTPS)
Exposes	1 service per port	1 service per LB	Many services via rules
Routing logic	None - port only	None - forwards everything	host/path-based
Cost	Free, but ugly ports (30000+)	1 cloud LB per service = certain amount price	1 LB total for everything
TLS	Handle it yourself	Handle it yourself	Built-in TLS termination

16. What is a StatefulSet? Explain how it differs from a Deployment and give one real-world use case.

A statefulset is a workload controller like deployment, but designed for applications that need a stable identity, pods that must keep the same name, network address, and storage across restarts.

The core difference is a deployment treats pods as interchangeable cattle: random names, any pod can be replaced by any other. But a statefulset treats pods as named individuals: if it dies, it comes back with the same name with same storage attached.

Real-world use case:

- For an example, name-0 is the primary (writes), name-1 and name-2 are read replicas
- Replicas connect to the primary at its fixed DNS name name-0.name – possible only because the name never changes.
- Each pod has its own PVC holding its data. If name-2 gets rescheduled to another node, kubernetes reattaches its volume – it resumes replication instead of starting from zero.
- Ordered startup ensures the primary is up before replicas try to connect

17. What is the Horizontal Pod Autoscaler (HPA)? Explain how it works and what metrics it uses to scale.

Horizontal Pod Autoscaler (HPA) is a controller that automatically adjusts the replica count of a deployment (statefulset, replicaset) based on observed load. Traffic spikes → more pods, load drops → fewer pods. It scale out/in , more/fewer pods, unlike the vertical pod autoscaler which scales up and down , bigger and smaller pods

How it works: the control loop

- Every 15 secs(default), the HPA controller queries metrics for the pods it targets.
- Metrics come from the metrics server for CPU/memory via metrics API – doesn't read pods directly. No metrics server installed = HPA shows unknown and does nothing.
- It computes like $\text{desired replicas} = \text{ceil}(\text{current replicas} \times \text{current metric} / \text{target metric})$
- It clamps the result between minimum replicas and maximum replicas and updates the workloads. The deployment controller does the actual pod creation
- Stabilization window (default 300s for scale-down)) prevents flapping, it won't kill pods the instant load dips.

18. Explain how ConfigMaps and Secrets are injected into a Pod (as environment variables and as mounted volumes).

Both the configmap and secret are key-value objects stored in etcd. Pods consume the two ways are as

- Environment variables
- Mounted volumes → keys become volumes,

The mechanics are identical for both secrets just add base64 encoding and stricter handling

19. Research any three **kubectl** commands that were not covered during the training and explain their purpose with one example each.

a. Kubectl top command

- Like the task manager for your cluster shows live CPU(millicores) and memory (Mi) usage
- Kubectl top pod-name

b. Kubectl cp command

- Copy files in and out of a container

```
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ kubectl cp ./service.yaml three-nginx-fd956d49d-qr766:/
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ kubectl exec -it three-nginx-fd956d49d-qr766 sh
error: exec [POD] [COMMAND] is not supported anymore. Use exec [POD] -- [COMMAND] instead
See 'kubectl exec -h' for help and examples
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ kubectl exec -it three-nginx-fd956d49d-qr766 -- bin/bash
root@three-nginx-fd956d49d-qr766:/# ls
bin  dev  docker-entrypoint.d  etc  home  lib64  mnt  proc  run  service.yaml  sys  usr
boot  docker-entrypoint.d  etc  lib  media  opt  root  sbin  srv  tmp  var
root@three-nginx-fd956d49d-qr766:/# |
```

-

c. Kubectl drain command

i. Empty a node so i can take it down

```
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
minikube      Ready     control-plane  4h52m  v1.35.1
minikube-m02  Ready     <none>      4h52m  v1.35.1
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ kubectl drain minikube-m02
Command 'kubectl' not found, did you mean:
  command 'kubectx' from deb kubectx (0.9.5-1ubuntu0.4)
Try: sudo apt install <deb name>
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ kubectl drain minikube-m02
node/minikube-m02 cordoned
error: unable to drain node "minikube-m02" due to error: [cannot delete Pods with local storage (use --delete-emptydir-data to override): kuberneteshdashboa
rd/dashboard-metrics-scraper-5565989548-rdhfc, kuberneteshdashboa/dashboard-kuberneteshdashboa-b84665fb8-6nwpf, cannot delete DaemonSet-managed Pods (use --ignore
-daemonsets to ignore): kube-system/kindnet-xj9zh, kube-system/kube-proxy-xvq9g], continuing command...
There are pending nodes to be drained:
  minikube-m02
cannot delete Pods with local storage (use --delete-emptydir-data to override): kuberneteshdashboa/dashboard-metrics-scraper-5565989548-rdhfc, kuberneteshdashboa/dashboard-kuberneteshdashboa-b84665fb8-6nwpf
cannot delete DaemonSet-managed Pods (use --ignore-daemonsets to ignore): kube-system/kindnet-xj9zh, kube-system/kube-proxy-xvq9g
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ kubectl drain minikube-m02 --ignore-daemonsets --delete-emptydir-data
node/minikube-m02 already cordoned
Warning: ignoring DaemonSet-managed Pods: kube-system/kindnet-xj9zh, kube-system/kube-proxy-xvq9g
evicting pod kuberneteshdashboa/dashboard-kuberneteshdashboa-b84665fb8-6nwpf
evicting pod default/three-nginx-fd956d49d-qr766
evicting pod kuberneteshdashboa/dashboard-metrics-scraper-5565989548-rdhfc
evicting pod default/three-nginx-fd956d49d-c7tmq
pod/kuberneteshdashboa-b84665fb8-6nwpf evicted
pod/three-nginx-fd956d49d-qr766 evicted
pod/three-nginx-fd956d49d-c7tmq evicted
pod/dashboard-metrics-scraper-5565989548-rdhfc evicted
node/minikube-m02 drained
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ |
```

ii.

20.Create a GitHub repository named week2-docker-k8s-assignment containing:

a. A Dockerfile and a built image pushed to Docker Hub (include the image link)

```
Login Succeeded
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/docker-files$ docker build -t dockervarun432:careerbytecode .
[*] Building 2.0s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 214B
=> [internal] load metadata for docker.io/library/nginx:alpine 0.0s
=> [auth] library/nginx:pull token for registry-1.docker.io 1.5s
=> [internal] load .dockerignore 0.0s
=> transferring context: 2B 0.0s
=> [internal] load build context 0.0s
=> transferring context: 31B 0.0s
=> [1/3] FROM docker.io/library/nginx:alpine@sha256:4a73873bd557c65b759505da037898b61f1be6bcc3c2c3aeac22d2a470c 0.0s
=> resolve docker.io/library/nginx:alpine@sha256:4a73873bd557c65b759505da037898b61f1be6bcc3c2c3aeac22d2a470c 0.0s
=> CACHED [2/3] COPY index.html /usr/share/nginx/html/ 0.0s
=> CACHED [3/3] RUN sed -i 's/listen 80;/listen 8080;/' /etc/nginx/conf.d/default.conf 0.0s
=> exporting to image 0.2s
=> exporting layers 0.0s
=> exporting manifest sha256:cbc35a115ae66261f1b9b893473f5fc109f100433d484bace92f281e99447e2 0.0s
=> exporting config sha256:39640a20f0a323cc4b3b9d5175f117243938c402353039c63bcd4dc326a3e665 0.0s
=> exporting attestation manifest sha256:8ec94bf209bb311c1aecd76d0645245350b7a0be5a0755a6af363e2e3e087eee 0.0s
=> exporting manifest list sha256:fe6cf3f141b218877f54c88a68ad55b16fd1b2cb4a88d911d9a66737d174ac6 0.0s
=> naming to docker.io/library/dockervarun432:careerbytecode 0.0s
=> unpacking to docker.io/library/dockervarun432:careerbytecode 0.0s
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/docker-files$ docker push dockervarun432:careerbytecode:latest
The push refers to repository [docker.io/dockervarun432:careerbytecode]
2d3064b8615b: Mounted from library/postgres
c86ab38aaddb: Mounted from library/postgres
0749aef11dca: Mounted from library/postgres
713473b5b92b: Mounted from library/postgres
2f537278bc74: Mounted from library/postgres
cc7fa208b8a7: Mounted from library/postgres
ef6ab46d98b2: Mounted from library/postgres
55afalecc21d: Mounted from library/postgres
4096c457dc0d: Mounted from library/postgres
689a434e3a86: Mounted from library/postgres
7af924000309: Mounted from library/postgres
latest: digest: sha256:7a396fd264a2067788b6551122b50f162bf6136312c7fc9d74381cb92c648382 size: 3054
Info > Not all multiplatform-content is present and only the available single-platform image was pushed
sha256:57c727d2a120e916c7fcd9995836d4d5f301e9408c0a05ef58f8df30ffc07777 -> sha256:7a396fd264a2067788b6551122b50f162bf6136312c7fc9d74381cb92c648382
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/docker-files$ |
```

Docker image link :

<https://hub.docker.com/repository/docker/dockervarun432/careerbytecode/general>

b. A Deployment and a Service YAML manifest

```
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ sudo vi deployment.yaml
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ kubectl apply -f deployment.yaml
deployment.apps/myapp-deployment created
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ kubectl get deployment.yaml
error: the server doesn't have a resource type "deployment"
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ kubectl get deployment
NAME          READY    UP-TO-DATE    AVAILABLE    AGE
myapp-deployment  0/3      3              0            12s
three-nginx     3/3      3              3            3h
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ kubectl get svc
NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
kubernetes    ClusterIP     10.96.0.1      <none>          443/TCP     7h1m
nginx-service  NodePort      10.100.183.17  <none>          80:30080/TCP 167m
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ |
```

c. A README.md explaining your setup

d. At least five Git commits

e. Screenshots of your Minikube deployment

```
varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ minikube status
minikube
type: Control Plane
host: Stopped
kubelet: Stopped
apiserver: Stopped
kubeconfig: Stopped

minikube-m02
type: Worker
host: Stopped
kubelet: Stopped

varun@DESKTOP-3JF00FS:/mnt/c/Users/admin/week-2-assignment/k8s-files$ minikube start --driver=docker --nodes 2
minikube v1.38.1 on Ubuntu 24.04 (kvm/amd64)
🔧 Using the docker driver based on existing profile
❗ You cannot change the number of nodes for an existing minikube cluster. Please use 'minikube node add' to add nodes to an existing cluster.
🔥 Starting "minikube" primary control-plane node in "minikube" cluster
📥 Pulling base image v0.0.50 ...
🔄 Restarting existing docker container for "minikube" ...
🔧 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
🔍 Verifying Kubernetes components...
   ▪ Using image docker.io/kubernetes/dashboard:v2.7.0
   ▪ Using image docker.io/kubernetes/metrics-server:v1.0.8
💡 Some dashboard features require the metrics-server addon. To enable all features please run:

    minikube addons enable metrics-server

🌟 Enabled addons: dashboard

🔥 Starting "minikube-m02" worker node in "minikube" cluster
📥 Pulling base image v0.0.50 ...
🔄 Restarting existing docker container for "minikube-m02" ...
🌐 Found network options:
   ▪ NO_PROXY=192.168.49.2
🔧 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
   ▪ env NO_PROXY=192.168.49.2
🔍 Verifying Kubernetes components...
🎉 Done! kubectrl is now configured to use "minikube" cluster and "default" namespace by default
```